Is It the Agent or the Pipeline? Attributing English–Bangla Failures of LLM Agents across Four Benchmarks

Anonymous ACL submission

Abstract

LLM agents are evaluated mostly in English. When they are evaluated in another language, a lower score is usually read as a weaker agent. But the score is produced by a pipeline: a translated task, sometimes a simulated user, tools and an automatic grader. We run two open-weight agents, `gemma4-31b` and `gpt-oss-20b`, on paired English and Bangla versions of four benchmarks (OfficeBench, WorkBench, τ^2-bench and TheAgentCompany) and trace every task that passes in English but fails in Bangla. The Bangla gap depends on model and benchmark: after Holm correction, `gemma4-31b` loses 8.0 points on OfficeBench and 11.5 on τ^2-bench; no other comparison is significant. Of 163 English-pass/Bangla-fail cases, the largest group, 74 (45.4%), traces to evaluation components: a Bangla-speaking simulated user, English-only checkers, translation or the environment. Between 14.7% and 54.6% of cases can be Bangla-specific agent failures, depending on how unclear and shared failures are counted. Crediting answers rejected only for their script cuts `gemma4-31b`'s OfficeBench gap from 8.0 to 3.0 points. The genuine failures on OfficeBench share one pattern: the agent keeps the meaning but writes the wrong form. Raw cross-lingual agent scores can overstate language-specific deficits.

1 Introduction

LLM agents plan over many steps, call tools and change files, databases and conversations (Yao et al., 2023; Liu et al., 2024; Zhou et al., 2024). Their benchmarks are almost all in English (Jimenez et al., 2024; Xie et al., 2024; Mialon et al., 2024). Recent work translates agent benchmarks into other languages and reports lower scores (Hofman et al., 2026; Wang et al., 2025a; Li et al., 2026). The usual reading is that the agent is weaker in that language.

That reading assumes the score measures only the agent. It does not. A score is produced by a pipeline: a translated task, sometimes a simulated user or simulated coworkers, tools, and an automatic grader. Any of these can fail in the new language. Two examples from our runs show this. On OfficeBench, `gemma4-31b` answers a Bangla question with the correct number written in Bengali digits; the checker searches for the ASCII string "40" and marks the run as failed. On τ^2-bench, the agent asks the Bangla-speaking simulated customer to confirm a cancellation; the customer confirms and ends the conversation in the same message, so the agent never performs the cancellation. Both runs count as Bangla failures. Neither is a failure of the agent.

We therefore ask where English–Bangla failures come from. We run two open-weight agents, `gemma4-31b` and `gpt-oss-20b`, on paired English and Bangla versions of four benchmarks that route the outcome through different components (Figure 1). For every task that passes in English but fails in Bangla, we compare the two trajectories and assign the failure to one cause. We address three questions:

- **RQ1.** How large is the English–Bangla gap, and does it depend on the model and the benchmark?
- **RQ2.** Where in the trajectory do Bangla-only failures first occur?
- **RQ3.** Which pipeline component is responsible for each Bangla-only failure?

Our contributions are:

- A paired English–Bangla evaluation of two agents on four benchmarks, with exact tests and bootstrap intervals (§4.1).
- A case-level attribution protocol with operational definitions and a fixed decision order, applied to all 163 English-pass/Bangla-fail cases (§3.5, §4.3).
- Evidence that the gap is model- and benchmark-

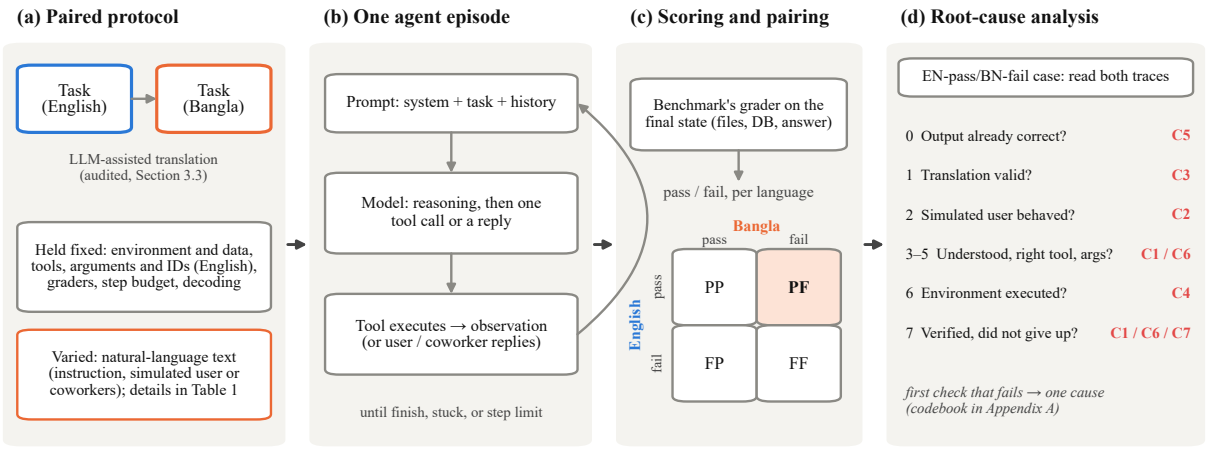


Figure 1: Evaluation protocol. (a) Each task runs once in English and once in Bangla; only natural-language text changes. (b) The agent loop. (c) The benchmark's grader scores each run, and the two runs of a task form one paired outcome. (d) Every English-pass/Bangla-fail case is traced and assigned the first cause whose check fails (codes in Table 4 and Appendix A).

specific and that the largest group of Bangla-only failures traces to evaluation components (§4.1–§4.3).

- Concrete recommendations for builders of multilingual agent benchmarks (§6).

2 Related Work

Agent benchmarks. Interactive benchmarks evaluate LLMs as agents (Liu et al., 2024; Zhou et al., 2024; Xie et al., 2024; Jimenez et al., 2024; Mialon et al., 2024). We use four with different outcome paths (§3): OfficeBench (Wang et al., 2025c), WorkBench (Styles et al., 2024), τ^2-bench (Barres et al., 2025), which extends τ-bench (Yao et al., 2024), and TheAgentCompany (Xu et al., 2024), run in OpenHands (Wang et al., 2025b).

Multilingual evaluation of LLMs and agents. Static multilingual benchmarks show an English advantage that grows for lower-resource languages (Conneau et al., 2018; Ahuja et al., 2023; Shi et al., 2023; Bandarkar et al., 2024; Huang et al., 2023; Zhang et al., 2023; Etxaniz et al., 2024). Recent agent benchmarks translate tasks and measure the gap: MAPS covers four agent suites in eleven languages (Hofman et al., 2026), and X-WebAgentBench, OmnilingualGAIA2, PolyWorkBench, SEATauBench and MASSIVE-Agents cover web, assistant, workplace, conversational and function-calling tasks (Wang et al., 2025a; Caciolai et al., 2026; Li et al., 2026; Nguyen et al., 2026; Kulkarni et al., 2025). These works quantify the size of the gap. We ask which component of the pipeline produces it.

Multilingual tool use. Agents must emit well-formed calls (Schick et al., 2023; Qin et al., 2024; Patil et al., 2024). In multilingual settings, models often write argument values in the user's language instead of the form the tool expects (Luo et al., 2026; Chauhan et al., 2026). These studies isolate argument-language errors in tool calling. We find a related form error inside full agent runs, and we separate it from errors of the checker and the simulated user.

Validity and failure attribution. The Agentic Benchmark Checklist (Zhu et al., 2025) and protocol-validity audits (Shao et al., 2026) show that setup and grading flaws can change conclusions. Failure taxonomies (Cemri et al., 2025) and diagnostic tools (Barke et al., 2026) analyse trajectories, and automatic attribution of the decisive step remains unreliable (53.5% agent-level and 14.2% step-level accuracy; Zhang et al., 2025). We apply manual, evidence-based attribution to cross-lingual failures and rule out evaluation causes before attributing a failure to the agent.

3 Methodology

3.1 Benchmarks and sampling

Table 1 summarises the four benchmarks. We use OfficeBench in full (300 tasks). For WorkBench we draw 15 tasks from each of its six domains with a fixed seed (90 of 690 tasks) . For τ^2-bench we

select tasks from each domain in their original order and drop every task whose English or Bangla run ended in an infrastructure or quota error. This leaves 218 paired tasks for `gemma4-31b` and 149 for `gpt-oss-20b`; 149 tasks are gradeable in all four cells . For TheAgentCompany we run 23 English–Bangla task pairs over 21 unique tasks; two tasks appear in both of our two task sets, and two further sampled tasks ran only in English and are excluded.

3.2 Bangla localization

In every benchmark the natural-language task is in Bangla, and everything the environment executes stays in English: tool names, argument keys, identifiers and the grader (Table 1). This keeps every action executable. It does not make language the only difference between the two runs. The translation itself can change a task, and on two benchmarks a simulated user or coworkers speak Bangla. We treat both as part of the Bangla condition and measure their effect in §4.3.

Translations were LLM-assisted. The intended rules were: keep names, URLs and file paths in Latin script; keep strings matched by a grader in English; write all other content, including numbers and dates, as a Bangla speaker would . Writing numbers in Bangla style and letting the agent answer in Bangla are part of the treatment, and both can cause English graders to reject correct work. For OfficeBench, the instructions were machine-translated by `gemma4-31b` and cached, so both agents saw the same string. An audit of the 298 translated instructions shows that the rules were applied unevenly: person names stayed in Latin script in 43 of 103 mentions, 44 instructions use Bengali digits and 50 use ASCII digits (14 use both), and 1 of 262 file names was altered.

3.3 Experimental setup

Both models were served through an Ollama endpoint with temperature 0 (Appendix B). Each benchmark runs with its own harness and limits: OfficeBench allows 20 steps per task, WorkBench uses single-turn function calling, τ^2-bench pairs the agent with a simulated customer played by the same model, and TheAgentCompany runs an OpenHands agent . Each task is run once per language with identical settings, so every comparison is paired at the task level; on TheAgentCompany, 13 Bangla tasks were also rerun, and we use the latest run. We log every instruction, model output, tool call, tool result, error and final answer.

For OfficeBench we repaired three defects in the shipped scorer: email checks now read the run's own files, mailbox lookup tries the exact account name first, and text files are read as UTF-8. No check was relaxed. The repairs raise the pass counts from 132, 103, 80 and 97 to 151, 127, 86 and 106 (`gemma4-31b` English and Bangla, `gpt-oss-20b` English and Bangla) and turn no pass into a fail.

3.4 Measuring the gap

Let EN and BN be the success rates in English and Bangla. We report the absolute gap $\Delta = \mathrm{EN} - \mathrm{BN}$ and the multilingual effectiveness loss rate, $\mathrm{MELR} = (\mathrm{EN} - \mathrm{BN})/\mathrm{EN}$, the share of English successes lost in Bangla. MELR is unstable when EN or the number of differing tasks is small, so we always report it with n. A *discordant pair* is a task with different outcomes in the two languages: an *EN-pass/BN-fail* (EN-only) or a *BN-pass/EN-fail* (BN-only) task. We test each gap with McNemar's exact test on the discordant pairs (McNemar, 1947), apply Holm correction across the seven model–benchmark comparisons (Holm, 1979), and report 95% paired bootstrap intervals (4,000 resamples of tasks). Subgroup tests are Holm-corrected within their family. Per-task-type rates (§5) are descriptive.

3.5 Attribution of Bangla-only failures

For every EN-pass/BN-fail case we read both trajectories side by side. We record the *first failure point*, the earliest step that, corrected alone, would most likely have produced a pass, and assign exactly one cause by checking, in order: (0) whether the Bangla output is already correct for the task as written; (1) whether the translation is valid; (2) whether the simulated user or coworker behaved correctly; (3–5) whether the agent understood the task and chose the right tool and arguments; (6) whether the environment executed a valid action; and (7) whether the agent verified its result and did not give up early (Figure 1d). The first check that fails decides the cause:

- **Checker** (C5): the output is correct for the task as written but the grader rejects it, or the check asks for something the task does not.
- **Translation** (C3): the Bangla text changed the task or a literal.
- **Simulated user** (C2): the Bangla simulated user or coworker broke the interaction.

	OfficeBench	WorkBench	τ^2-bench	TheAgentCompany
Interaction	multi-step	single-turn	dialogue	long-horizon
Simulated	–	–	customer	coworkers
Grading	file checks	DB state	DB state, actions, device	checkpoints
Sample	full	15 per domain	ordered, per domain	subset
Paired n	300	90	218 / 149	23 (21 tasks)
Models	both	both	both	`gemma4-31b`
In Bangla	system prompt, task	system prompt, task, reply	scenario, policy, agent prompt	task, coworker replies
Kept English	tools, IDs, files, graders	tools, IDs, dates	IDs, product names, enums	apps, files, URLs

Table 1: Benchmarks. Paired n: tasks run and graded in both languages (τ^2-bench: `gemma4-31b` / `gpt-oss-20b`). The localization rows describe what was translated in each benchmark .

- **Tool / environment** (C4): a harness or tool fault blocked a valid action.
- **Genuine Bangla failure** (C1): an agent error that the English run does not show.
- **General weakness** (C6): an agent error whose mechanism also appears in an English failure of the same model, cited by task.
- **Unclear** (C7): no visible mechanism, or Bangla reruns of the task disagree.

Checker–agent boundary. A difference only in script or numerals (Bengali digits, a Bangla word for a requested English keyword) is a checker artefact. A value changed into a different English form ("curd" for *yogurt*, "08:00 AM" for *8:00*) is an agent failure (one relabelling this implies is noted in Appendix A).

Reliability. Labels follow a written codebook (Appendix A). A second annotator reviewed all WorkBench cases, 46 of 66 τ^2-bench cases and 16 OfficeBench cases; agreement on the fine cause ranges from fair ($\kappa = 0.39$) to substantial ($\kappa = 0.78$); details with n are in Appendix A.

4 Results

4.1 Size of the Bangla Gap

RQ1: *How large is the English–Bangla gap, and does it depend on the model and the benchmark?*
Finding. The Bangla gap depends on both the model and the benchmark. Only `gemma4-31b` loses significantly after correction, on OfficeBench (8.0 points) and τ^2-bench (11.5 points); `gpt-oss-20b` is higher in Bangla on OfficeBench, and neither model changes on WorkBench.

Table 2 gives every comparison. The two `gemma4-31b` losses rest on 42 EN-only against 18 BN-only tasks on OfficeBench (50.3% → 42.3%) and 40 against 15 on τ^2-bench (49.5% → 38.1%). `gpt-oss-20b`'s OfficeBench gain (28.7% → 35.3%, 25 against 45 tasks) is significant before correction ($p = .022$) but not after. On TheAgentCompany, `gemma4-31b` drops from 39.1% to 26.1%, but this rests on three discordant tasks, so its MELR of 33.3% is not informative; over the 21 unique tasks the gap is 9.5 points (2 against 0 tasks).

The two models also differ from each other. On OfficeBench, `gemma4-31b`'s gap exceeds `gpt-oss-20b`'s by 14.7 points (95% CI [7.7, 22.3]). On the 149 τ^2-bench tasks gradeable in all four cells, the gaps are $+13.4$ and -0.7 points, a difference of 14.1 (95% CI [2.0, 26.2]). The model gap itself depends on the language. On OfficeBench in English, `gemma4-31b` solves 82 tasks that `gpt-oss-20b` fails and `gpt-oss-20b` 17 that `gemma4-31b` fails ($p < 10^{-10}$); in Bangla the counts are 48 and 27 ($p = .020$).

`gpt-oss-20b`'s Bangla gain should not be read as better understanding. Its invalid actions fall from 1,671 in English to 955 in Bangla over 299 runs each (Table 6): the Bangla prompt changes how well it follows OfficeBench's action format.

On OfficeBench, `gemma4-31b`'s gap is concentrated in short tasks: 16.1 points on single-app tasks (18 against 3 discordant tasks, Holm-adjusted $p = .004$), 9.5 on two-app tasks (16 against 7, $p = .186$) and 0.0 on three-app tasks (8 against 8). Three-app tasks fail in both languages for the same reasons, so they carry little language signal.

	Model	n	EN	BN	Δ [95% CI]	MELR	b/c	p_{H}
OB	gemma	300	50.3	42.3	+8.0 [3.0, 13.0]	15.9	42/18	**.016**
	gpt-oss	300	28.7	35.3	−6.7 [−12.0, −1.3]	−23.3	25/45	.112
WB	gemma	90	81.1	82.2	−1.1 [−10.0, 7.8]	−1.4	7/8	1.00
	gpt-oss	90	58.9	53.3	+5.6 [−5.6, 16.7]	9.4	15/10	1.00
τ^2	gemma	218	49.5	38.1	+11.5 [5.0, 17.4]	23.1	40/15	**.007**
	gpt-oss	149	38.9	39.6	−0.7 [−10.7, 8.7]	−1.7	26/27	1.00
TAC	gemma	23	39.1	26.1	+13.0 [0.0, 26.1]	33.3	3/0	1.00

Table 2: Success rates (%) in English and Bangla on OfficeBench (OB), WorkBench (WB), τ^2-bench (τ^2) and TheAgentCompany (TAC). Positive Δ and MELR mean Bangla is worse. b/c: EN-only/BN-only tasks. p_{H}: exact McNemar test, Holm-corrected across the seven comparisons; bold: $p_{\mathrm{H}} < .05$.

	OB		τ^2		WB		TAC
First failure point	g	o	g	o	g	o	g
Task level	18	13	17	16	5	2	2
Tool level	6	3	10	3	2	12	2
Control	3	6	12	4	0	1	3
Final answer only	15	3	1	0	0	0	0
Unclear	0	0	0	3	0	0	1
Cases	42	25	40	26	7	15	8

Table 3: First failure point of the analysed cases (g: gemma, o: gpt-oss). Task level: understanding, grounding, constraints or planning; tool level: tool choice, arguments or execution; control: verification or early stop. TheAgentCompany cases include Bangla reruns that disagree and both-fail tasks with different partial scores.

4.2 Where Bangla Runs Break

RQ2: *Where in the trajectory do Bangla-only failures first occur?*

Finding. Each benchmark breaks at a different point: Bangla-only failures start at the final answer on OfficeBench, in the dialogue on τ^2-bench, at tool arguments on WorkBench, and at verification and stopping on TheAgentCompany.

Table 3 gives the counts. On OfficeBench, 15 of `gemma4-31b`'s 42 Bangla-only failures appear only at the final response, after every earlier step matched the English run. On τ^2-bench, failures start in grounding, planning and termination (11 of `gemma4-31b`'s 40 at an early stop). On WorkBench, 11 of `gpt-oss-20b`'s 15 cases start at tool arguments: missing filters, wrong thresholds or invented argument names. On TheAgentCompany, the analysed cases break at verification and stopping. No Bangla-only τ^2-bench failure starts with malformed tool syntax, and none of the 641 WorkBench tool-call arguments we inspected contains Bangla script.

4.3 Causes of Bangla-Only Failures

RQ3: *Which pipeline component is responsible for each Bangla-only failure?*

Finding. The largest group of Bangla-only failures, 74 of 163 cases (45.4%), traces to evaluation components rather than the agent, and only 14.7–54.6% can be Bangla-specific agent failures. Correcting the grading cuts `gemma4-31b`'s OfficeBench gap from 8.0 to 3.0 points.

Table 4 attributes all 163 analysed cases. These are cases, not shares of the gap: we did not apply the same codebook to every BN-pass/EN-fail task, so the table does not say how much of each gap is artefactual. The evaluation-side cases come from the simulated user (35, all on τ^2-bench), checkers that reject correct answers (22, almost all on OfficeBench), translation (9) and tools or the environment (8). Agent causes account for 69 cases (42.3%) and 20 are unclear. The Bangla-specific share is 14.7% as labelled, 27.0% with the unclear cases and 54.6% with the 45 general-weakness cases, whose label rests on one English analogue per case that single runs cannot confirm.

The dominant cause follows the benchmark design: English-only checkers on OfficeBench, the simulated customer on τ^2-bench and general weakness on WorkBench, where no simulated user is involved and the grader compares database state. TheAgentCompany has too few cases for a dominant cause (3 general, 3 unclear, of which 2 are Bangla reruns that disagree).

Stage and cause differ. Each case has a first failure point (§4.2) and a cause, and the two coincide only in part (Table 9). Failures at the final answer are checker artefacts (18 of 19). Only 10 of the 35 simulated-user cases are early stops (10 of `gemma4-31b`'s 11 on τ^2-bench); 20 start at task level, after the simulated customer gave a wrong value or changed a constraint. Tool-level failures are mostly agent-caused (30 of 38), and genuine Bangla failures start at grounding (13), understanding or planning (3) or tool use (8), never at control or the final answer. Where a run breaks does not show whose fault it is.

Cause	OB	τ^2	WB	TAC	All (%)
Evaluation side					
Simulated user (C2)	0	35	0	0	35 (21.5)
Checker (C5)	21	0	1	0	22 (13.5)
Translation (C3)	6	1	1	1	9 (5.5)
Tool / env. (C4)	7	0	0	1	8 (4.9)
Agent side					
Genuine Bangla (C1)	13	8	3	0	24 (14.7)
General (C6)	4	21	17	3	45 (27.6)
Unclear (C7)	16	1	0	3	20 (12.3)
Cases	67	66	22	8	163

Table 4: Cause of every analysed EN-pass/BN-fail case, both models pooled per benchmark. Evaluation-side causes account for 74 cases (45.4%), agent causes for 69 (42.3%) and unclear cases for 20 (12.3%).

	Model	As scored	Grading corr.	Confounds removed
OB	gemma	+8.0	+3.0 [−1.3, 7.3]	+1.9 [−1.9, 5.6]^{s}
	gpt-oss	−6.7	−8.7 [−13.7, −3.3]	−3.3 [−7.7, 1.5]^{s†}
WB	gemma	−1.1	−2.2 [−10.1, 5.6]^{s}	−3.4 [−11.4, 4.5]^{s}
	gpt-oss	+5.6	+6.7 [−3.4, 18.0]^{s}	+8.0 [−2.3, 18.2]^{s}
τ^2	gemma	+11.5	+10.6 [4.1, 17.0]	+5.8 [−1.9, 12.8]^{s}
	gpt-oss	−0.7	−0.7 [−10.7, 8.7]	−0.7 [−10.5, 9.1]^{s}
TAC	gemma	+13.0	+8.7 [0.0, 21.7]	0.0 [0.0, 0.0]^{s}

Table 5: Gap (points, 95% CI) under corrected grading (OB, TAC: credit checker cases; τ^2: credit Bengali-digit answers; WB: drop the ambiguous task) and with evaluation-side cases removed. ^{s}: both languages (OB English: seven environment traps). †: also drops 19 English action-format failures.

Corrected grading and confound removal. Table 5 recomputes the gaps under two corrections. Crediting answers that fail only on script or digits (on OfficeBench, the 15 `gemma4-31b` answers with correct content) cuts `gemma4-31b`'s OfficeBench gap from 8.0 to 3.0 points, and its interval then includes zero. Removing evaluation-side cases moves `gemma4-31b`'s τ^2-bench gap from 11.5 to 5.8 points when dialogues that the simulated user ended before the agent acted are dropped in *both* languages. The corrections that touch only the Bangla side push the remaining gaps down by construction, so we mark which ones are symmetric.

Two genuine Bangla mechanisms. On OfficeBench, all 13 genuine Bangla cases follow two mechanisms, and both appear in both models. In *entity grounding*, a name that appears only in Bangla script has no letter case; the agent writes it in lower case for a case-sensitive mailbox or calendar lookup, receives an empty result and accepts it (3 `gemma4-31b` and 6 `gpt-oss-20b` cases; the same two tasks fail for both models). In *re-rendering*, the agent turns a value from the Bangla task into a different English form: "Trip" for *travelling* (both models), "curd" for *yogurt*, and "08:00 AM" for *8:00*. The same pattern appears once on WorkBench, where "last week" is grounded as a rolling seven-day window. In each case the meaning survives and the form the environment requires does not. The eight genuine τ^2-bench cases are more varied: grounding of names and identifiers, calls to user-side device tools in telecom, and policy checks the English run performed.

	gemma		gpt-oss	
Measure (OfficeBench)	EN	BN	EN	BN
Invalid actions (299 runs)	273	194	1,671	955
Runs with ≥3 identical rejected actions in a row	9	9	60	12
Failed step repeated identically	11%	16%	17%	7%
Writing runs that read back output	3/180	4/178	20/112	19/155
Out-of-step failures on 3-app tasks	38/46	30/39	49/81	37/58
of which no step was rejected	13/46	21/39	0/81	1/58
of which ≥half the steps were rejected	9/46	10/39	44/81	9/58
Bangla-script final answer	–	46/173	–	7/194

Table 6: Trajectory measurements on OfficeBench, computed from the run logs. Failed step repeated: share of rejected or erroring steps followed by an identical action (284, 213, 1,640 and 949 failed steps). Read back: a written file is read again, or a created event is followed by listing the calendar. Last row: share of Bangla failures.

4.4 Quantified trajectory observations

We measure step-level behaviour on OfficeBench, the one benchmark where it is defined identically for both models and not shaped by a simulated user: every step is a parsed text action, so a rejected action is a logged event, and both models share a 20-step budget over the largest paired sample (1,200 runs). Table 6 reports the measures; stage counts cover all failures.

The two models fail at different stages. An agent step moves from understanding to planning, action encoding, execution, verification and completion. `gpt-oss-20b` often fails at action encoding: 107 of its 214 English OfficeBench failures produce no valid action or give up (54 coded as action format), against 12 of `gemma4-31b`'s 149. On WorkBench, where function calling handles the format, its failures move to the arguments: 40 of its 79 failed runs are wrong tool arguments, against 11 of `gemma4-31b`'s 33. `gemma4-31b` usually reaches execution and fails later, on content (43 of 149 English failures), the step budget (42 of 149) and, in Bangla, the script of its answer. The model gap follows the same line: of the 82 English tasks that `gemma4-31b` solves and `gpt-oss-20b` does not, 80 `gpt-oss-20b` runs contain a rejected action, 33 are coded action format, and 42 end stuck or at the step limit.

Running out of steps means different things. Out-of-step failures are mostly a three-app problem (38 of 46 for `gemma4-31b` in English). For `gemma4-31b`, 13 of these 46 runs have no rejected step at all: valid progress was too slow. For `gpt-oss-20b`, none of 81 is of this kind, and in 44 at least half of the steps were rejected, so the budget was spent on invalid actions.

Verification and repetition. The agents rarely check what they wrote: `gemma4-31b` reads back its output in under 3% of writing runs, `gpt-oss-20b` in 12–18%. `gpt-oss-20b`'s English runs contain 1,671 invalid actions and 60 runs with three or more identical rejected actions in a row; both fall sharply in Bangla. One common claim is *not* supported: most failed steps are not followed by an identical action (7–17% are), so repetition is concentrated in a minority of runs. Whether missing verification *causes* failures is a hypothesis (§6).

Other benchmarks. These measures do not transfer: on WorkBench one invalid call fails the run outright, on τ^2-bench the simulated user co-produces the trajectory, and TheAgentCompany has 23 task pairs for one model, so we report run-level signals. On τ^2-bench, besides early stops by the simulated customer, the most frequent signal is a required write never made (`gemma4-31b`: 39/110 English and 37/135 Bangla failures; `gpt-oss-20b`: 34/91 and 43/90), then writes with wrong arguments (19–25 per cell); one run reaches the step limit, and Bangla runs use fewer tool calls ($7.69 \rightarrow 6.64$; $7.26 \rightarrow 6.44$). On WorkBench, Bangla runs are also shorter ($3.96 \rightarrow 3.17$ and $3.30 \rightarrow 2.93$ tool calls per task), and `gpt-oss-20b` crashes runs by inventing argument names (2 English, 5 Bangla). The eight traced TheAgentCompany cases show early stops and unchecked success claims but are too few to count.

4.5 Evaluation-setup failures in both languages

RQ3 counts only EN-pass/BN-fail cases. The evaluation setup also fails on tasks that fail in both languages, which lowers both scores and can hide or mimic agent errors.

Scorer and benchmark defects (OfficeBench). The three scorer defects repaired in §3 turned 19 English and 24 Bangla runs of `gemma4-31b` (6 and 9 of `gpt-oss-20b`) from fail to pass; the shipped scorer therefore understates both languages, Bangla slightly more. Beyond the scorer, some checks ask for something the task does not: a benchmark defect is the primary cause of 47 of `gemma4-31b`'s 149 English failures (31.5%) and 39 of the 131 tasks that fail in both languages, and it decides at least one of the three fully coded runs on 57 tasks. In task 1-2/0, all four runs give the correct free time slot and all four fail, because two of the four checks require a calendar event the task never asks for. Defects do not make a task uninformative: 18 of the 19 `gpt-oss-20b` English runs whose primary cause is a defect (12 of 47 for `gemma4-31b`) also contain rejected actions, so dropping these tasks would also remove evidence about the agents.

Simulated users (τ^2-bench). A heuristic detector flags 46 `gemma4-31b` Bangla dialogues and 26 English ones in which the simulated customer ended the chat before the agent acted (a hand check of six random flags found about three true early stops). In 33 `gemma4-31b` and 54 `gpt-oss-20b` Bangla dialogues, the simulated customer wrote mostly in English, a fidelity problem the agent cannot fix.

Harness and environment limits. On OfficeBench, the in-container LLM helper answers in at most 50 tokens; for `gpt-oss-20b`, whose hidden reasoning uses that budget, all 14 helper calls in two Bangla runs returned empty strings.

The reply-length cap rejected 14 and 15 consecutive long writes in two `gemma4-31b` Bangla runs (Appendix E), and two runs ended after empty model replies. On TheAgentCompany, cloud-drive authentication errors and an administrator password prompt blocked valid actions in both languages. Provider errors do not drive the gaps: dropping every WorkBench task with a provider error on either side leaves `gpt-oss-20b`'s gap at $+7.1$ points ($n = 85$).

5 Analysis by Task Type

We assigned every task of WorkBench, τ^2-bench and TheAgentCompany to one of nine types (Appendix D, Table 10) . The results are descriptive (several types have fewer than ten tasks; no per-type tests). Three patterns hold for types with at least ten tasks. First, failure rates differ more between types than between languages: on τ^2-bench, multi-target operations fail 73–93% of the time in every cell ($n = 44$ and 27). Second, the largest increases are on τ^2-bench, where the simulated customer speaks Bangla: single-target tasks for `gemma4-31b` ($35\% \rightarrow 62\%$, $n = 34$), entity creation for `gpt-oss-20b` ($55\% \rightarrow 82\%$, $n = 11$) and conditional decisions for `gemma4-31b` ($27\% \rightarrow 43\%$, $n = 37$). On WorkBench, `gemma4-31b`'s failure count differs by at most two tasks per type. Third, the two models can move in opposite directions within a type: conditional decisions on τ^2-bench change by $+16$ points for `gemma4-31b` and -10 for `gpt-oss-20b` ($n = 37$ and 30), and system setup and troubleshooting by $+6$ and -11 ($n = 87$ and 46). The opposite movement of the two models in §4.1 therefore persists inside task types and is not only a product of task mix.

6 Discussion

Implications for benchmark builders. Our results suggest five practices. (1) Separate checks of artefact state from checks of answer text, and accept the task language or normalise numerals in answer checks; otherwise correct Bangla answers fail. (2) Constrain simulated users: forbid confirming and ending a dialogue in the same turn, and require device actions to be tool calls. (3) Protect names, literals and identifiers in translation, and audit the result automatically. (4) Report discordant counts, corrected-grading gaps and per-case attribution alongside raw scores. (5) Report results by benchmark and model; a pooled "Bangla penalty" hides opposite effects.

Implications for agent builders. Untested scaffold changes follow from the same failures: map common tool aliases, re-prompt after a rejected action, allow bulk spreadsheet edits, require a read-back before finishing, keep a checklist of subgoals, and copy literals verbatim.

Hypotheses for future work. Seven explanations fit our data but are not demonstrated by it; each names the experiment that would test it. (H1) *Missing verification causes failures*: require a read-back before finishing. (H2) *Long tasks fail by error accumulation*: vary the step budget on three-app tasks. (H3) *Scores measure model–scaffold compatibility*, and interface reliability rather than reasoning limits `gpt-oss-20b`: rerun it with native function calling in both languages. (H4) *Script and tool-format errors share one cause*, a correct intent in a rejected form: use a probe set with protected names and literals. (H5) *Early errors are rarely recovered*: prompt a retry after an empty result and measure recovery. (H6) *Control can fail when the plan is right* (querying after a usable answer, retrying instead of switching): label whether the reasoning at the decisive step names the correct action. (H7) *The scaffold forces plan reconstruction*, since the OfficeBench harness passes earlier actions but not earlier reasoning: carry the reasoning forward and measure repeated actions and missed constraints.

7 Conclusion

We ran paired English and Bangla evaluations of two open-weight agents on four benchmarks. The Bangla gap depends on model and benchmark: only two of seven comparisons are significant after correction. Among the 163 tasks that fail only in Bangla, the largest group traces to evaluation components, and correcting the grading removes most of one significant gap. The genuine failures on OfficeBench keep the meaning of the task but get its form wrong. Cross-lingual agent scores should be read together with an attribution of their failures.

Limitations

Case-level attribution. We attribute cases, not gaps. On OfficeBench and τ^2-bench the BN-pass/EN-fail tasks were not coded with the same codebook (WorkBench is the exception), so the

corrected gaps that drop cases only on the Bangla side are biased toward zero.

Single runs. Each task ran once per language at temperature 0. Served inference is not bit-deterministic: rerunning 13 Bangla TheAgentCompany tasks changed full completion for one and the checkpoint score for three. The general-weakness label assumes that one English analogue reflects a stable weakness.

Trajectory measures. Step-level measures (rejected actions, repetition, read-back, use of the step budget) come from OfficeBench only, for the reasons given in §4.4. The other benchmarks are described by run-level signals, so the two kinds of evidence are not directly comparable, and our step-level findings may not generalise to dialogue or function-calling harnesses.

Annotation. Causes were assigned by one annotator per benchmark, with second review on WorkBench (all cases), τ^2-bench (46 of 66) and a 16-case check on OfficeBench. Agreement on the fine cause is fair to substantial. Some annotators were LLMs, which may share biases with the evaluated agents.

Translation. Bangla tasks were machine-translated with LLM assistance. On OfficeBench, the translation rules were applied unevenly (§3.2), so some failures we attribute to the agent may partly reflect translation. Native-speaker review of the translations is incomplete.

Scope. We study one language, two models and pilot-sized samples of WorkBench (90 tasks) and TheAgentCompany (23 task pairs). On τ^2-bench the two models have different task sets after infrastructure exclusions, and the simulated user is played by the same model as the agent.

Ethical Considerations

We evaluate public open-weight models on public benchmarks in sandboxed environments; no human subjects or private data are involved. English-only graders can penalise an agent for answering in the user's language, which understates agent quality for non-English users. We release task IDs, trajectories, attribution labels and analysis scripts so that this bias can be measured. The Bangla tasks were machine-translated and are not a curated Bangla language resource.

References

Kabir Ahuja, Harshita Diddee, Rishav Hada, Millicent Ochieng, Krithika Ramesh, Prachi Jain, Akshay Nambi, Tanuja Ganu, Sameer Segal, Maxamed Axmed, Kalika Bali, and Sunayana Sitaram. 2023. MEGA: Multilingual evaluation of generative AI. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Lucas Bandarkar, Davis Liang, Benjamin Muller, Mikel Artetxe, Satya Narayan Shukla, Donald Husa, Naman Goyal, Abhinandan Krishnan, Luke Zettlemoyer, and Madian Khabsa. 2024. The Belebele benchmark: a parallel reading comprehension dataset in 122 language variants. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*.

Shraddha Barke, Arnav Goyal, Alind Khare, Avaljot Singh, Suman Nath, and Chetan Bansal. 2026. AgentRx: Diagnosing AI agent failures from execution trajectories. In *Proceedings of the 43rd International Conference on Machine Learning (ICML)*.

Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik Narasimhan. 2025. τ^2-bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*.

Andrea Caciolai, Pere-Lluís Huguet Cabot, Chierh Cheng, Albert Ventayol-Boada, Gabriel Mejia Gonzalez, Christophe Ropers, Lucas Bandarkar, Sebastian Ruder, Darlene Sakakihara, Elliot Yun, Pierre Andrews, Grégoire Mialon, Romain Froger, and Marta R. Costa-jussà. 2026. OmnilingualGAIA2: Evaluating the multilingual gap in frontier AI agents. *arXiv preprint arXiv:2608.08775*.

Mert Cemri, Melissa Z. Pan, Shuyi Yang, et al. 2025. Why do multi-agent LLM systems fail? In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*.

Siddharth Chauhan, Thomas Butler, Abhishek Singhania, Pankaj Porwal, and Honey Gupta. 2026. When the API speaks the wrong language: Revisiting post-training for multilingual tool use. *arXiv preprint arXiv:2608.11715*.

Alexis Conneau, Ruty Rinott, Guillaume Lample, Adina Williams, Samuel R. Bowman, Holger Schwenk, and Veselin Stoyanov. 2018. XNLI: Evaluating cross-lingual sentence representations. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Julen Etxaniz, Gorka Azkune, Aitor Soroa, Oier Lopez de Lacalle, and Mikel Artetxe. 2024. Do multilingual language models think better in English? In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*.

Omer Hofman, Jonathan Brokman, Oren Rachmil, Shamik Bose, Vikas Pahuja, Toshiya Shimizu,

Trisha Starostina, Kelly Marchisio, Seraphina Goldfarb-Tarrant, and Roman Vainshtein. 2026. MAPS: A multilingual benchmark for agent performance and security. In *Findings of the Association for Computational Linguistics: EACL 2026*.

Sture Holm. 1979. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, 6(2):65–70.

Haoyang Huang, Tianyi Tang, Dongdong Zhang, Wayne Xin Zhao, Ting Song, Yan Xia, and Furu Wei. 2023. Not all languages are created equal in LLMs: Improving multilingual capability by cross-lingual-thought prompting. In *Findings of the Association for Computational Linguistics: EMNLP 2023*.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. 2024. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations (ICLR)*.

Mayank Kulkarni, Vittorio Mazzia, Judith Gaspers, Chris Hench, and Jack FitzGerald. 2025. MASSIVE-Agents: A benchmark for multilingual function-calling in 52 languages. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 20193–20215.

J. Richard Landis and Gary G. Koch. 1977. The measurement of observer agreement for categorical data. *Biometrics*, 33(1):159–174.

Hongliang Li, Yijin Liu, Zhiwei Zhang, Zihe Liu, Xinyue Lou, Jinan Xu, Fandong Meng, and Kaiyu Huang. 2026. PolyWorkBench: Benchmarking LLM agents for cross-lingual long-horizon workflows. *arXiv preprint arXiv:2607.06008*.

Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, and 3 others. 2024. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations (ICLR)*.

Zheng Luo, T Pranav Kutralingam, Ogochukwu N. Okoani, Wanpeng Xu, Hua Wei, and Xiyang Hu. 2026. Lost in execution: On the multilingual robustness of tool calling in large language models. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 44059–44077.

Quinn McNemar. 1947. Note on the sampling error of the difference between correlated proportions or percentages. *Psychometrika*, 12(2):153–157.

Grégoire Mialon, Clémentine Fourrier, Craig Swift, Thomas Wolf, Yann LeCun, and Thomas Scialom. 2024. GAIA: A benchmark for general AI assistants. In *International Conference on Learning Representations (ICLR)*.

My Chiffon Nguyen, Aulia Adila, Saksorn Ruangtanusak, Kittiphat Leesombatwathana, Vissuta Gunawan Lim, Patomporn Payoungkhamdee, and Samuel Cahyawijaya. 2026. SEATauBench: Progressively adapting tool-agent-user evaluation into low-resource Southeast Asian languages. *arXiv preprint arXiv:2606.28715*.

Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. 2024. Gorilla: Large language model connected with massive APIs. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, Sihan Zhao, Lauren Hong, Runchu Tian, Ruobing Xie, Jie Zhou, Mark Gerstein, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ToolLLM: Facilitating large language models to master 16000+ real-world APIs. In *International Conference on Learning Representations (ICLR)*.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Jiaqi Shao, Hanck Chen, Wei Zhang, Maxm Pan, and Bing Luo. 2026. Do agent benchmarks measure capability? protocol validity in the age of agentic AI. *arXiv preprint arXiv:2607.22368*.

Freda Shi, Mirac Suzgun, Markus Freitag, Xuezhi Wang, Suraj Srivats, Soroush Vosoughi, Hyung Won Chung, Yi Tay, Sebastian Ruder, Denny Zhou, Dipanjan Das, and Jason Wei. 2023. Language models are multilingual chain-of-thought reasoners. In *International Conference on Learning Representations (ICLR)*.

Olly Styles, Sam Miller, Patricio Cerda-Mardini, Tanaya Guha, Victor Sanchez, and Bertie Vidgen. 2024. WorkBench: a benchmark dataset for agents in a realistic workplace setting. In *Conference on Language Modeling (COLM)*.

Peng Wang, Ruihan Tao, Qiguang Chen, Mengkang Hu, and Libo Qin. 2025a. X-WebAgentBench: A multilingual interactive web benchmark for evaluating global agentic system. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 19320–19335.

Xingyao Wang, Boxuan Li, Yufan Song, Frank F. Xu, et al. 2025b. OpenHands: An open platform for AI software developers as generalist agents. In *International Conference on Learning Representations (ICLR)*.

Zilong Wang, Yuedong Cui, Li Zhong, Zimin Zhang, Da Yin, Bill Yuchen Lin, and Jingbo Shang. 2025c. OfficeBench: Benchmarking language agents across multiple applications for office automation. In *Findings of the Association for Computational Linguistics: NAACL 2025*. ArXiv:2407.19056.

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Lu, Kai Tan, Yanbo Shen, Zhennan Liu, Yuchen Guo, Zhiwen Wang, Yiheng Qi, et al. 2024. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments. In *Advances in Neural Information Processing Systems (NeurIPS)*.

Frank F. Xu, Yufan Song, Boxuan Li, Yuxuan Tang, et al. 2024. TheAgentCompany: Benchmarking LLM agents on consequential real world tasks. *arXiv preprint arXiv:2412.14161*.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.

Shaokun Zhang, Ming Yin, Jieyu Zhang, Jiale Liu, Zhiguang Han, Jingyang Zhang, Beibin Li, Chi Wang, Huazheng Wang, Yiran Chen, and Qingyun Wu. 2025. Which agent causes task failures and when? on automated failure attribution of LLM multi-agent systems. In *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, volume 267 of *Proceedings of Machine Learning Research*.

Xiang Zhang, Senyu Li, Bradley Hauer, Ning Shi, and Grzegorz Kondrak. 2023. Don't trust ChatGPT when your question is not in english: A study of multilingual abilities and types of LLMs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.

Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, Robert Lo, Abishek Sridhar, Xianyi Cheng, Tianyue Ou, Yonatan Bisk, Daniel Fried, Uri Alon, and Graham Neubig. 2024. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations (ICLR)*.

Yuxuan Zhu, Tengjun Jin, Yada Pruksachatkun, Andy Zhang, Shu Liu, Sasha Cui, Sayash Kapoor, et al. 2025. Establishing best practices in building rigorous agentic benchmarks. In *Advances in Neural Information Processing Systems, Datasets and Benchmarks Track*.

A Attribution codebook

Checker–agent rule. One TheAgentCompany case labelled translation (coworkers reply in Bangla and a checkpoint needs the English word "helpful") is a checker artefact under this rule; moving it would change the translation count from 9 to 8 and the checker count from 22 to 23, with the evaluation-side total unchanged.

Code	Operational definition
C5 Checker	The Bangla output is correct for the task as written; the grader rejects it, or the check requires something the task does not ask for. Includes script- or numeral-only differences.
C3 Translation	The Bangla task text differs from the English task in an operation, a constraint or a literal (e.g., a quoted string translated; "sum" rendered as "add").
C2 Simulated user	The simulated user or coworker ends the interaction early, gives a wrong value, replies in the wrong language, or narrates an action instead of performing it.
C4 Tool / env.	A tool, harness or infrastructure fault blocks an action that is valid (e.g., an empty model reply recorded as malformed; a reply cap that rejects a long valid action).
C1 Genuine Bangla	The agent errs in a way the English run does not, and no check above fails (e.g., a changed English value, a lower-cased name from Bangla script).
C6 General	The agent error's mechanism also appears in an English failure of the same model, cited by task ID.
C7 Unclear	No visible mechanism, or Bangla reruns of the same task disagree.

Table 7: Causes, checked in the order C5, C3, C2, then agent understanding, tool choice and arguments, C4, and verification (Figure 1d). The first check that fails decides the cause. Each case also records the first failure point, the decisive action and a trace quote.

Agreement. On WorkBench, a blind second annotator labelled every case and disagreements were adjudicated (cause $\kappa = 0.59$, $n = 7$, `gemma4-31b`; $\kappa = 0.77$, $n = 15$, `gpt-oss-20b`). On τ^2-bench, 46 of 66 cases were double-coded and adjudicated ($\kappa = 0.39$ on the fine cause, fair agreement on the scale of Landis and Koch, 1977; $\kappa = 0.71$, substantial, on evaluation versus agent). On OfficeBench, the codebook was checked on 16 double-coded cases ($\kappa = 0.78$); the remaining cases and all TheAgentCompany cases have one annotator. The WorkBench and TheAgentCompany annotators were LLMs .

B Reproducibility details

C First failure point by cause

D Task types

E Example failure chains

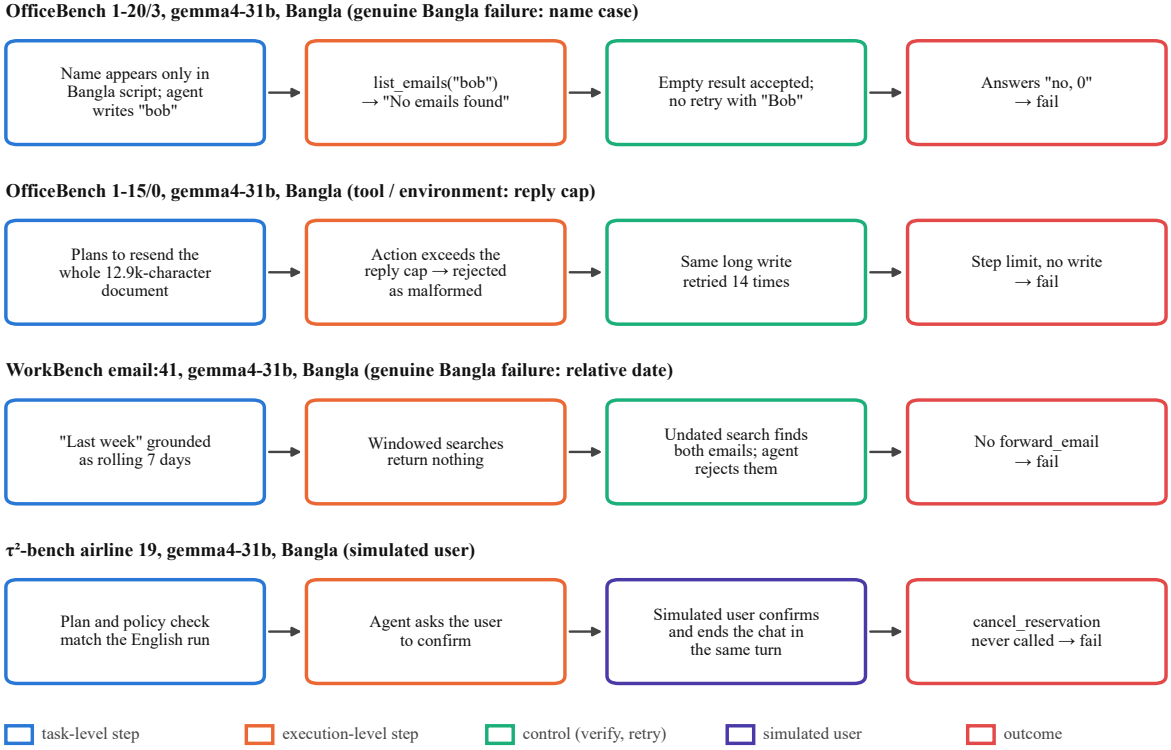


Figure 2: Four traced Bangla runs whose English counterpart passes, from the first divergence to the final failure. These are illustrations; the counts are in Tables 4 and 6.

Item	Value
Model tags	`gemma4:31b`, `gpt-oss:20b`
Serving	Ollama, OpenAI-compatible endpoint
Decoding	temperature 0, top-p 1 (OfficeBench harness); `gpt-oss-20b` reasoning effort not set (default)
Reply cap (OfficeBench)	512 tokens (`gemma4-31b`), 2,048 (`gpt-oss-20b`)
Step budgets	OfficeBench 20 steps; TheAgentCompany ; WorkBench, τ^2-bench
Simulated users	τ^2-bench: same model as the agent; TheAgentCompany coworkers
Translation	OfficeBench: `gemma4-31b`, cached; system prompt written in Bangla; others LLM-assisted
OfficeBench scorer	three repairs (§3); scripts `analysis/fa_rescore.py`
Statistics	exact McNemar, Holm, paired bootstrap (4,000 resamples, seed 7)
Annotation	codebook in Appendix A; LLM annotators on WorkBench and TheAgentCompany

Table 8: Configuration. Fill in every value that the run logs and reports do not record before submission.

First failure point	C1	C6	C2	C5	C3	C4	C7	All
Task level	16	17	20	4	7	2	7	73
Tool level	8	22	1	0	0	6	1	38
Control	0	6	12	0	0	0	11	29
Final answer only	0	0	0	18	1	0	0	19
Unclear	0	0	2	0	1	0	1	4
All	24	45	35	22	9	8	20	163

Table 9: First failure point (rows, as in Table 3) against cause (columns, Table 4) for all 163 EN-pass/BN-fail cases, from the per-case trace notes. C1 genuine Bangla, C6 general weakness, C2 simulated user, C5 checker, C3 translation, C4 tool or environment, C7 unclear.

	WB gemma		WB gpt-oss		τ^2 gemma		τ^2 gpt-oss		TAC gemma	
Task type	EN/BN	n	EN/BN	n	EN/BN	n	EN/BN	n	EN/BN	n
Entity creation	0/0	8	38/25	8	67/73	15	55/82	11	50/75	4
Single-target	0/0	23	13/26	23	35/62	34	71/74	34	100/100	3
Multi-target and batch	24/16	25	56/48	25	73/80	44	81/93	27	–	
Temporal reasoning	40/40	5	60/80	5	–		–		100/100	1
Data analysis and QA	0/14	7	14/57	7	0/100	1	0/0	1	100/100	4
Conditional decisions	42/42	19	63/68	19	27/43	37	37/27	30	100/100	2
Cross-tool composition	33/33	3	33/33	3	–		–		–	
Interpersonal coord.	–		–		–		–		40/60	5
System setup and troubl.	–		–		53/59	87	61/50	46	0/25	4
All tasks	19/18	90	41/47	90	50/62	218	61/60	149	61/74	23

Table 10: Failure rate (%) in English/Bangla per task type, with the number of tasks. Descriptive: with fewer than ten tasks one task moves a rate by 10–100 points.

Type	What the task asks
Entity creation	Make one new record from details in the request
Single-target	Find the one record the request points to, then change it
Multi-target	Select every matching record, or make several changes
Temporal	Work out free slots, relative dates or time shifts
Data analysis	Compute, chart or look up an answer from data
Conditional	Check a condition or policy first; act only if it holds
Cross-tool	Use a value from one tool as input to another
Interpersonal	Obtain information or agreement from coworkers
System setup	Install software or walk a user through a fix

Table 11: The nine task types.